10Power IoT Fellowship
in collaboration with **NESL** (New Energy Solutions Lab)

Continuous Residual-Current (Leakage) Detection for Distributed Energy Management

Integrating the IVY MD0630 Type-B RCM into the OpenAMI / MeshEMS firmware: from Modbus bring-up to multi-EMS isolation and reverse-flow-adaptive protection

Consolidated technical report: sprints S1 to S6, with DC bench validation and vendor protocol confirmation

Author
Allahaddje Bonheur
IoT Fellow, 10Power
ESP32 firmware and embedded integration

Technical supervision
Glenn, NESL

Target platform
NESL EMS controller (ESP32-S3, PCB 865B)
Firmware branch `modbus-testing`

August 2026

Summary

This report summarises the work carried out over six sprints (S1 to S6) to integrate the **IVY MD0630 Type-B AC+DC leakage sensor** (a residual-current monitor, RCM) into the **OpenAMI / MeshEMS** EMS firmware. The sprints were:

- **S1**: bring up the Modbus link and derive the register map.
- **S2**: read the live AC/DC leakage on the ESP32.
- **S3**: write the safety thresholds, always confirmed by a read-back.
- **S4**: add an energy-change correlation cache.
- **S5**: build a multi-EMS nomination and MQTT isolation scheme.
- **S6**: sense the hardware alarm outputs and adapt the AC threshold under reverse power flow.

Two later steps close the loop: a **DC bench-injection test** that confirms detection, corrects the current scaling and demonstrates the alarm threshold, and **IVY's official protocol reply** that independently validates the derived register map and settles the open questions on leakage direction and alarm signalling.

Each sprint below gives its objective, method and result. The work followed the **"path of least blockage"** approach: build with configurable parameters and a mock pipeline so progress never stalls on an unknown, then confirm each assumption on real hardware.

Table of contents

Nomenclature

Term	Meaning
AC / DC	Alternating / direct current component of the residual current
CRC	Cyclic redundancy check (Modbus frame integrity)
EMS	Energy Management System (the NESL ESP32-S3 controller)
FC	Modbus function code (FC03 read, FC06 write-single, FC16 write-multiple)
GPIO	General-purpose input/output pin
LV	Low voltage
MQTT	Message Queuing Telemetry Transport (publish/subscribe messaging)
PV / EV	Photovoltaic / electric vehicle
RCD / RCM	Residual-current device (protective) / monitor (measuring)
RCM Type B	Monitor that senses AC and DC residual current
RS-485 / UART	Serial physical / logic layers

1 Context and objectives

The **IVY MD0630** is a Type-B residual-current monitor: it measures both the AC and the DC component of the earth-leakage current. A Type-B device is used on sites with power electronics (PV, battery storage, EV charging), where a DC residual current can appear and would not be seen by an ordinary AC-only RCD.

The sensor is a subsystem of the NESL **Energy Management System (EMS)**, an ESP32-S3 controller (PCB 865B) that meters energy, drives relays and speaks MQTT within the OpenAMI / MeshEMS platform. Because several EMS can share one low-voltage feeder, the leakage can be handled in a coordinated, multi-node way.

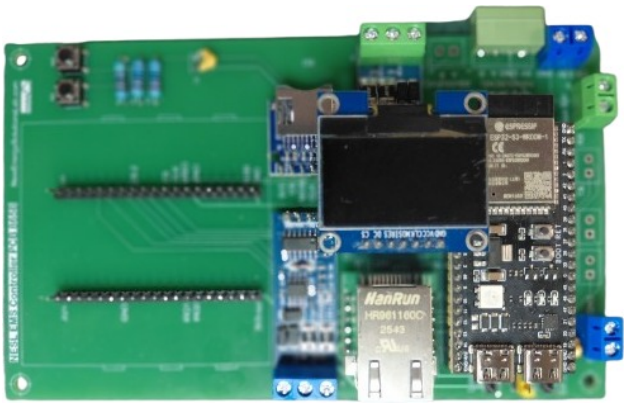

Figure 1: The NESL Street-EMS controller, the class of ESP32-S3 device into which the leakage subsystem is integrated.

Objective. Integrate the Type-B leakage monitor into the EMS firmware so the platform reads and acts on the residual current continuously. The work was split into the six sprints of Table 2.

Table 2: The six sprints of the assignment.

Sprint	Objective	Outcome
S1	Modbus bring-up and register map	Map derived, confirmed vs vendor tool
S2	Read live AC/DC leakage on the EMS	12/12 reads, 0 failures
S3	Write safety thresholds, with read-back	Solved via FC16 (not FC06); verified
S4	Correlate leakage changes with energy	Cache implemented and exercised
S5	Multi-EMS nomination and MQTT isolation	Proven on device and over MQTT
S6	Hardware alarm sense and reverse-flow threshold	Implemented; alarm path verified on HW

2 S1: Modbus bring-up and register map

Objective. Establish Modbus communication with the module and derive its register map (offsets, function code, scaling), which the datasheet does not give. This was done first on a PC bench, before committing the EMS bus.

Method. The module was powered at 12 V and read through a USB-TTL adapter as the Modbus master. The detail that unblocked communication was electrical: the module's UART is open-drain and needs pull-ups, and (because the lab adapter had no VCC wire) those pull-ups were fed from an ESP32 5 V pin, with every ground tied to a single common ground (Figure 2). Once the link was up, the register block was found by scanning, then every field was cross-checked against IVY's own `CT.exe` tool.

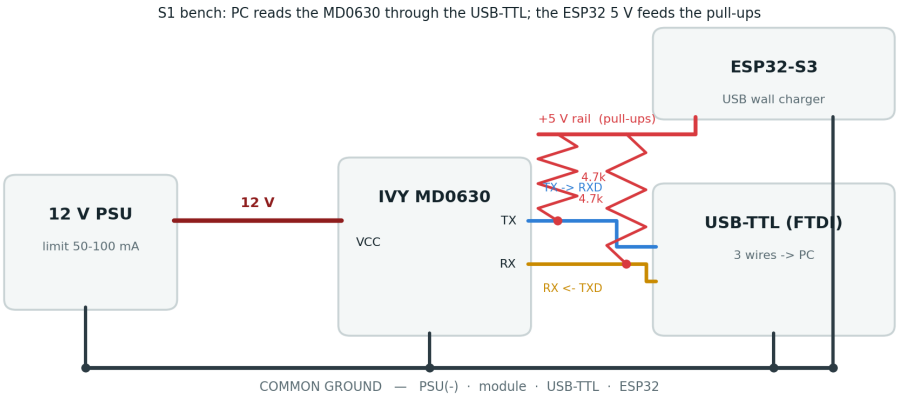

Figure 2: S1 bench wiring: 12 V powers the module; the PC reads it over the USB-TTL; the ESP32 5 V feeds the pull-ups; every ground is common.

Table 3: Derived MD0630 register map (FC 0x03). The leakage-current registers scale as raw × 0.01 mA and the threshold registers as raw × 0.1 mA. Defaults match the datasheet: DC 6 mA, AC 30 mA.

Register	Default (raw / value)	Field	Access
`0x0000`	0 / 0.00 mA	DC leakage current (raw × 0.01 mA)	R
`0x0001`	0 / 0.00 mA	AC leakage current (raw × 0.01 mA)	R
`0x0002`	60 / 6.0 mA	DC threshold (raw × 0.1 mA)	R/W
`0x0003`	300 / 30.0 mA	AC threshold (raw × 0.1 mA)	R/W
`0x0100`	1	Modbus slave address	R/W
`0x0111`	5	Firmware version	R

At this stage both leakage registers read 0 (no leakage at rest), so the current scaling could not yet be distinguished; it was later established as raw × 0.01 mA by DC injection and confirmed by IVY (see the *Bench validation* and *Vendor confirmation* sections). The threshold scaling (raw × 0.1 mA) was fixed directly from the `CT.exe` frames (`0x003C` = 6.0 mA, `0x012C` = 30.0 mA).

S1 result

Communication established at **address 1, 9600 8N1, FC 0x03**, and the register map (3) was derived from the live device and confirmed against the vendor's `CT.exe` frames. Leakage currents scale as raw × 0.01 mA and thresholds as raw × 0.1 mA.

3 S2: Live AC/DC read on the EMS

Objective. Have the EMS itself read the live residual current over its own serial bus, and feed the values into the firmware's leakage model (published on the `subpanel_RCMleaks` topic).

Method. The MD0630 uses plain UART (not RS-485 differential), so it connects directly to the ESP32 UART pins, no transceiver (Table 4). Pull-ups go to 3.3 V, since the ESP32-S3 GPIOs are not 5 V-tolerant.

Table 4: S2 bench wiring, direct UART.

Module	ESP32-S3	Note
TX	GPIO42 (RS485_1 RX)	+ 4.7 kΩ pull-up to 3V3
RX	GPIO7 (RS485_1 TX)	+ 4.7 kΩ pull-up to 3V3
GND	GND	common ground (EMS, module, 12 V PSU−)
VCC	12 V PSU	never the ESP32

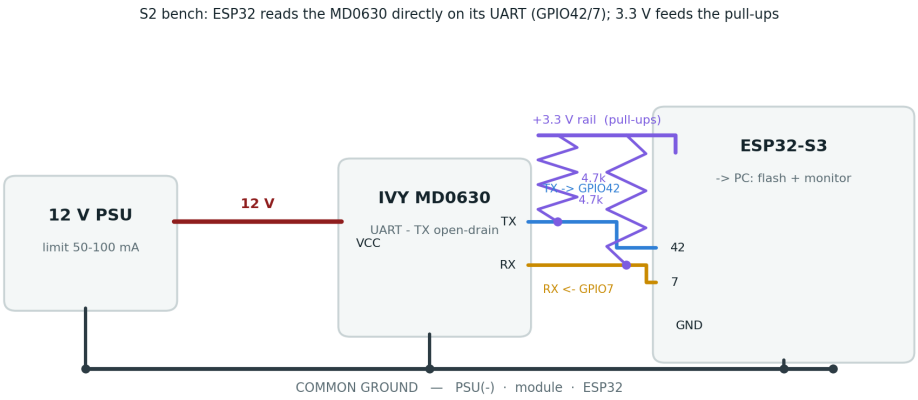

Figure 3: S2 bench wiring: the EMS reads the MD0630 directly on its UART (GPIO42/7); the module runs on 12 V; the ESP32 3.3 V feeds the pull-ups; every ground is common.

Two issues were found and fixed: the data wires were first landed on the board's silkscreen "RX/TX" (the USB console UART, GPIO44/43) instead of the Modbus bus (GPIO42/7); and the on-board SHT20 sensor, also at address 1, collided with the CT on the bus until a bring-up flag skipped its poll.

S2 result

The EMS reads the physical module reliably: **12 consecutive successful reads, 0 failures**, both channels 0.00 mA at rest. The S1 register map is validated on the target hardware.

4 S3: Config-time threshold writes

Objective. Write the safety thresholds (DC 6 mA at `0x0002`, AC 30 mA at `0x0003`) at commissioning time, always confirmed by a read-back, and never from the polling loop.

Method and finding. The first attempt used **FC 0x06 (write single register)**. The module accepted it (no Modbus exception) but silently ignored it: the register did not change, and only the read-back caught it. Sniffing the vendor tool doing a write showed the reason:

```
Write : 00 10 00 03 00 01 02 01 0E 2B A7   (FC 0x10, reg 0x0003, qty 1, value 0x010E = 270
Read  : 00 10 00 03 00 01 F0 18            (success; no unlock frame anywhere)
```

The MD0630 requires **FC 0x10 (write multiple registers)**, and there is no unlock step. The driver's `writeThreshold_mA()` now writes via FC16 and enforces a read-back:

Table 5: The write is never trusted without a matching read-back.

Step	Action
1	`setTransmitBuffer(0, mA × 10)`, e.g. 30 mA → 300 = `0x012C`

Step	Action
2	`writeMultipleRegisters(reg, 1)`, FC 0x10 to `0x0002` (DC) or `0x0003` (AC)
3	40 ms settle, then read back (FC 0x03)
4	Confirm: read-back == written → OK; else reject (`0xE4`)

On the EMS this swept AC 30 → 27 → 30 mA with matching read-backs, then restored the safe factory defaults (DC 6 / AC 30 mA).

S3 result

Threshold writes work via **FC 0x10 with a mandatory read-back**, verified on the EMS. The blocker was the function code (FC06 vs FC16), found by tracing the vendor tool, not an unlock step.

Following the agreed safety rules, writes are config-time only, target only the two threshold registers, always read back and reject on mismatch, and the vendor tool's factory Calibration function is never used.

5 S4: Energy-change correlation

Objective. Add a cache so that, whenever the leakage steps up or down, the system can look back over recent energy activity and record what energy event happened at that moment. This is the input the S5 manager consumes.

Method. Two streams meet in the cache (Figure 4). The EMS energy (`readings[0]`) is recorded continuously into a ring buffer of 128 timestamped snapshots, each deriving the active-power ramp against the previous sample. The leakage reading drives a step detector that fires when the change is at least 1 mA; on a step it queries the ring for the nearest snapshot within a 2 s window and attaches it to the event.

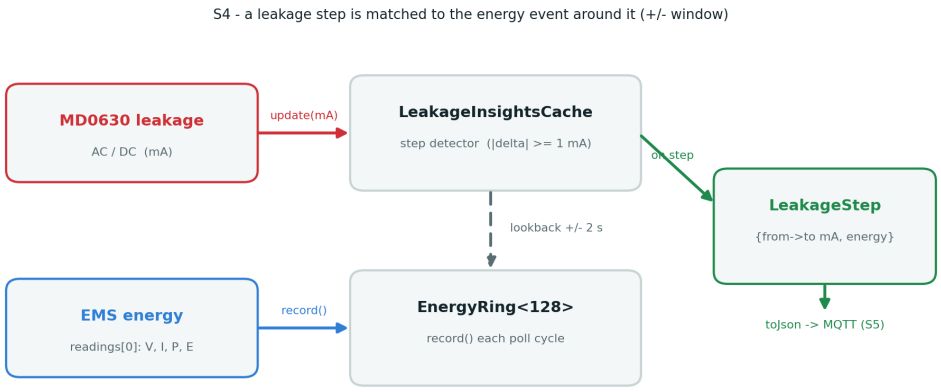

Figure 4: S4 data flow: a leakage step is matched to the energy event around it (within a time window).

The design is change-based, not level-based (it acts on steps, not the always-present standing residual), and the energy ring is generic so the same pattern is reusable for other insights. A `phase_angle_deg` field is reserved for a future RCM upgrade that would give leakage direction.

S4 result

An energy-change correlation cache (`EnergyRing`, `LeakageInsightsCache`) links each leakage step to the surrounding energy event. It compiles and is exercised end to end by the mock ramp.

6 S5: Multi-EMS nomination and MQTT isolation

Objective. When several EMS share a feeder, a single leak is seen (at different magnitudes) by several of them. S5 decides which EMS is nearest the fault and how to isolate it. The scheme is grounded in the fault-location literature (references [1] and [2]).

Method. Nomination ranks candidates by magnitude and by lowest peer-correlation (the "odd one out", the node whose ramp signature correlates least with its peers), and a confidence gate decides between auto-isolation (bracket the two EMS around the leak) and alert-only. The detection triggers on change, not on absolute level. A residual-voltage / residual-current phase-angle criterion is noted as the basis for a future direction feature, tied to the reserved `phase_angle_deg` field. IVY has since confirmed (see *Vendor confirmation*) that the MD0630 outputs the absolute residual current only, with no polarity or phase-angle register (per IEC 62955); leakage direction therefore requires an external residual-voltage reference or a directional RCM, not this sensor alone.

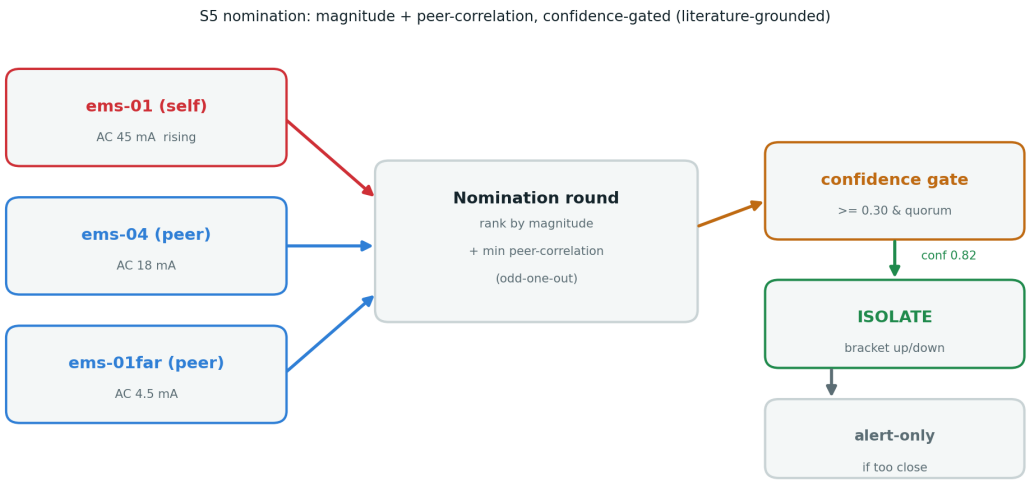

Figure 5: S5 nomination flow: rank by magnitude and by minimum peer-correlation, confidence-gated to auto-isolate or alert-only.

Evidence. The `LeakageInsightsManager` implements a per-EMS state machine (`normal`, `watch`, `isolating`, `fault`, `restored`) and the three MQTT payloads. Run on one ESP32 with two simulated peers (mid $\approx 0.4\times$, far $\approx 0.1\times$), the nominated node is both the magnitude leader and the least-correlated node, giving confidence 0.82 and an auto-isolate decision. The leakage was then streamed over MQTT to a broker on the PC over a real WiFi network, and charted live as the mock ramp rose and held at its ceiling (figures below).

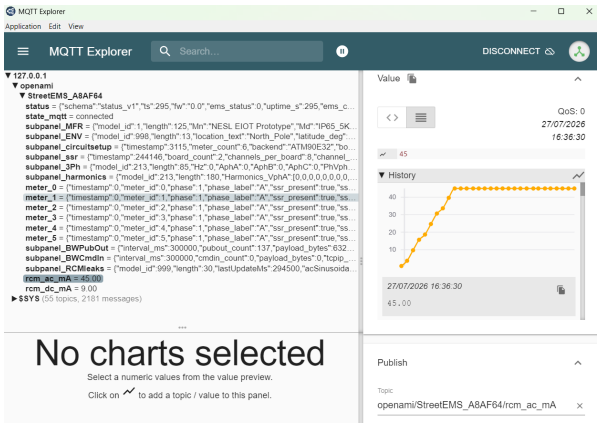

Figure 6: AC leakage rising to 45 mA then holding.

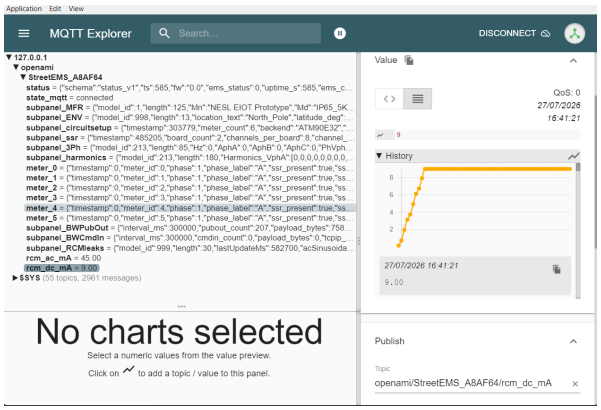

Figure 7: DC leakage rising to 9 mA then holding.

Figure: Live leakage telemetry charted in MQTT Explorer (`rcm_ac_mA` / `rcm_dc_mA`).

S5 result

The nomination (magnitude and peer-correlation, confidence-gated) locates a leak across three EMS on one ESP32, and the live leakage streams over MQTT with a fault flag and live charts. A WPA3/PMF hotspot blocked the ESP32's WiFi stack; a WPA2 hotspot connected immediately.

7 S6: Hardware alarm sense and reverse-flow threshold

S6 has two parts (Figure 8): the EMS observes the module's own hardware alarm (A), and it adapts the AC threshold to the site's power-flow direction (B).

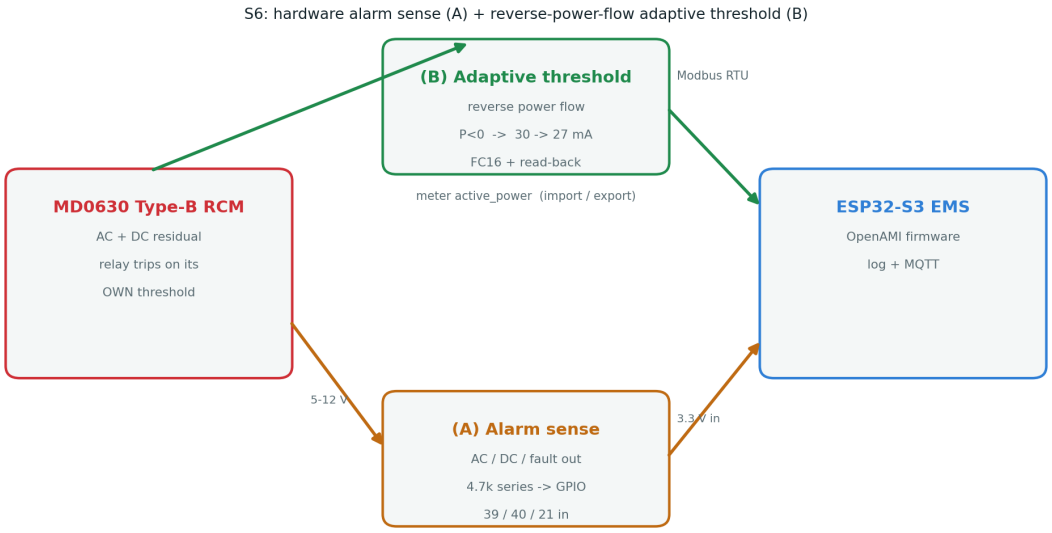

Figure 8: S6 architecture: hardware alarm sense (A) and reverse-power-flow adaptive threshold (B) between the MD0630 and the EMS.

(B) Reverse-power-flow adaptive threshold. Under export (PV or battery back-feed), the AC limit is tightened from 30 to 27 mA and restored on return to import. Export is detected from the meter's active power (below 50 W of back-feed). The threshold is re-written only on a debounced transition (three poll cycles), never every cycle, reusing the S3 FC16 write with read-back. The export flag is also passed to the S5 manager, which de-rates confidence under reverse flow.

(A) Hardware alarm sense. The MD0630 trips its own internal relay when leakage crosses a threshold. S6-A lets the EMS observe those AC/DC/fault alarm outputs so a trip is logged and published; it is read-only and never drives the trip. Each 5 V output reaches a GPIO through a single 4.7 kΩ series resistor, which is safe for any output type (an open-collector reads low when active; a push-pull 5 V high is clamped by the GPIO's internal ESD diode, with the resistor limiting the current to about 0.36 mA). IVY later confirmed the module exposes no Modbus status or self-test register, so these hardware outputs are the only way to observe a trip; the read-only sense of S6-A is therefore the correct and only approach.

During bench testing, three issues were found and fixed:

Table 6: S6-A issues found and fixed during hardware testing.

Issue found on hardware	Cause	Fix
Pins GPIO 33/34 absent on the devkit	not broken out on the ESP32-S3-DevKitC-1	move to GPIO 39/40/21
False `AC=1 DC=1` at rest	outputs are active-high (low at rest, high on alarm)	`active_low=false`, `INPUT_PULLDOWN`
Manual pulse missed by the logger	alarm sampled only at the slow Modbus poll	fast service at about 50 Hz

After the fixes, the rest state reads quiet and a stimulus on the AC line is logged live:

```
SETUP: MODBUS: MD0630 alarm sense on GPIO 39/40/21 (AC/DC/fault, active-high)
S6 ALARM edge: AC=0 DC=0 FAULT=0     (at rest, no leakage, quiet)
S6 ALARM edge: AC=1 DC=0 FAULT=0     (AO asserted, 3V3 pulse on GPIO39)
S6 ALARM edge: AC=0 DC=0 FAULT=0     (released; DC and FAULT stay 0)
```

S6 result

Both parts are implemented and gated behind build flags (the firmware builds cleanly with S6 off and with every S6 path on). **S6-A was verified on hardware**: the module alarm output reaches the GPIO through the 4.7 kΩ resistor and the firmware logs `AC=1` within 200 ms. S6-B reuses the S3 write path.

8 Bench validation: DC leakage injection

Objective. Confirm on real hardware that the EMS detects an *induced* DC leakage, verify the register map and scaling with a known current, and observe the DC alarm threshold being crossed.

Method. Glenn's resistor-ladder emulator injects a known DC current through the sensor window. A single wire from the ESP32 3.3 V rail passes once through the CT toroid, then

returns through a parallel resistor ladder to the common ground; each branch adds a known current by Ohm's law, so the current the CT sees is the sum of the active branches.

Table 7: DC bench injection: read values match Ohm's law to about 1 percent, all inside the 2 to 15 mA sensor range.

Active branches	Injected (Ohm's law)	Read DC leakage	AC channel
2 branches	5.06 mA	5.13 mA	0.00 mA
3 branches	7.13 mA	7.19 mA	0.00 mA
4 branches	10.13 mA	10.18 mA	0.00 mA

Findings.

- **Detection works end to end**: the induced DC leakage is read to about 1 percent.
- **Register map confirmed**: the DC injection moved `0x0000` while `0x0001` (AC) stayed at 0.00 mA, fixing DC = `0x0000` and AC = `0x0001` (the one assumption left open in S1).
- **Current scaling corrected**: the readings match only under raw × 0.01 mA, not × 0.1 mA (× 0.1 would imply 50 to 100 mA, impossible in the 2 to 15 mA range). The firmware now applies one scale to the currents (× 0.01 mA) and another to the thresholds (× 0.1 mA).
- **Threshold crossing shown**: 5.13 mA reads below the 6 mA DC threshold (no alarm), 7.19 mA reads above it (alarm), demonstrating the trip point.

Bench result

The DC bench injection is the first confirmed non-zero *induced* leakage on real hardware. It validates DC detection, confirms the DC/AC register assignment, corrects the current scaling to raw × 0.01 mA, and demonstrates the 6 mA DC threshold being crossed.

9 Vendor confirmation of the protocol

After the map was derived and the bench test done, IVY Metering returned an official, point-by-point confirmation (12 August 2026) that validates the reverse-engineered protocol and answers the remaining open questions.

- **Register map and scaling confirmed**: `0x0000` DC and `0x0001` AC leakage at raw × 0.01 mA per LSB; `0x0002` DC and `0x0003` AC thresholds at raw × 0.1 mA per LSB (`0x003C` = 6.0 mA, `0x012C` = 30.0 mA); `0x0100` slave address (R/W), `0x0111` firmware version. Communication at address 1, 9600 8N1, FC 0x03.
- **Write rule confirmed**: only FC 0x10 (write multiple) takes effect, FC 0x06 is ignored, and no unlock is required, exactly as found in S3.
- **Direction**: the module outputs the *absolute* residual current only, with no register for polarity, sign or phase angle (per IEC 62955). Leakage direction is not observable from this sensor; it would need an external residual-voltage reference or a directional RCM.
- **Alarm signalling**: there is no Modbus status or self-test register. A trip is available only on the hardware DO / AO / DA pins, which is exactly what the S6-A sense path reads.
- **Part number**: the communication-enabled variant is the **MD0630T41A**; per IVY the `-1` suffix omits serial communication. Our marked unit does communicate over Modbus, so any reorder for the fleet should specify the communication-enabled variant explicitly, a point to confirm with the vendor.

Vendor confirmation

IVY's official protocol reply (12 August 2026) independently validates the derived register map, the two scaling factors and the FC 0x10 write rule, and settles the open questions: the sensor is absolute-value only (no direction, IEC 62955) and exposes alarms only on its hardware pins.

10 Results summary

Table 8 collects the verification status of each capability. It distinguishes *code-proven* (exercised by the mock pipeline) from *hardware-proven* (validated against the physical module).

Table 8: Verification status across S1 to S6, with bench and vendor validation.

Capability	Verification	Status
Modbus link and register map (S1)	live device and vendor `CT.exe` frames	hardware-proven
Live AC/DC read (S2)	12/12 reads, 0 failures on the EMS	hardware-proven
Threshold write and read-back (S3)	AC 30/27/30 mA on the EMS; FC16 traced	hardware-proven
Energy-change correlation (S4)	compiles; step to correlation via mock ramp	code-proven
Nomination and MQTT isolation (S5)	3-EMS round on one ESP32; live MQTT and charts	hardware-proven (mock peers)
Reverse-flow threshold (S6-B)	builds; transition-only FC16 write path	code-proven
Hardware alarm sense (S6-A)	live `AC=1`/`AC=0` on the EMS, within 200 ms	hardware-proven
DC bench injection	5.1/7.2/10.2 mA vs Ohm's law; 6 mA threshold crossed	hardware-proven
Protocol confirmed by vendor	IVY point-by-point reply, 12 Aug 2026	vendor-confirmed

The firmware builds cleanly both in the default configuration (all leakage test flags off, no regression) and with every S1 to S6 capability compiled in, at about 19 % RAM and 28 % flash on the ESP32-S3. All source, drivers, per-sprint reports and diagrams are version-controlled on the `modbus-testing` branch.

References

1. *Faulty feeder identification in a distribution network by data analysis and similarity (trend-correlation) comparison of zero-sequence transients*, PMC7795609 (open access).
2. *FLISR: Fault Location, Isolation and Service Restoration*, workflow and practice.
3. *Additional criterion for faulty-feeder selection during ground faults using the residual voltage and residual current phase angle (about 90 degrees).*
4. Modbus Organization, *MODBUS Application Protocol Specification*, V1.1b3.

5. IEC 60755, *General requirements for residual current operated protective devices*; IEC 62955 (residual DC detection for EV charging).
6. IVY Metering, *MD0630 Type-B AC+DC residual-current monitor*, datasheet (register table on page 10) and `CT.exe` tool (vendor).
7. IVY Metering, official register-map and protocol confirmation (correspondence), 12 August 2026.